



Tunis Business School

Data Mining Project

E-commerce behavior analysis

Submitted By:

Maha Gharsalli
Rihem Abdelmoumen
Rayen Ghourabi

Major:BA Minor: IT

April, 2026

<https://canva.link/9zbfu93wmo2bv24>

TABLE OF CONTENTS

1	Executive Summary.....	2
2	Dataset Description	2
3	Problem Statement	4
4	Data Preparation	5
5	Methods And Analysis	15
6	Results And Interpretations	37
7	Conclusion	58

1. Executive Summary:

This report presents a data mining analysis of user behavior on an e-commerce platform. The dataset contains 8000 simulated user records capturing browsing and engagement metrics with 14 variables spanning demographic information, device usage, behavioral engagement metrics and marketing interaction signals with Purchase as the binary target variable indicating whether a given user completed a transaction during their session with the primary objective of predicting whether a user will make a purchase.

This project applies supervised classification techniques mainly Logistic Regression, Decision Tree, and Random Forest models to identify the most influential features driving conversion, evaluate predictive performance, and derive actionable insights for e-commerce decision-makers.

Data preparation steps include handling missing values (~2% of rows), encoding categorical variables, scaling numerical features, and splitting the data into training and test sets.

The results are intended to support customer conversion optimization strategies helping e-commerce platforms better understand and target their users.

2.Dataset Description:

2.1 Industry & Organizational Context:

The dataset belongs to the **e-commerce industry**.

The organizations considered are likely **online shopping platforms** (similar to large-scale e-commerce websites) that collect and analyze user interaction data to better understand customer behavior and improve business performance. The dataset represents **an e-commerce environment** where users browse products, interact with advertisements, and make purchasing decisions, with data typically processed by **a digital marketing analytics team** to monitor user engagement and optimize business strategies. The primary objective is to understand user behavior, predict purchasing decisions, improve conversion rates, and optimize marketing campaigns. This dataset is commonly used by data analysts, marketing teams, business intelligence professionals, and data scientists to support data-driven decision-making and enhance customer experience.

2.2 Source:

Platform: Kaggle

Dataset Name: E-Commerce Behavior Dataset (8000 Users)

URL:<https://www.kaggle.com/datasets/asifxzaman/e-commerce-behavior-dataset8000-users>

1.3 Observation Representation & Main Variables :

The dataset captures behavioral signals from 8,000 e-commerce sessions. Each row represents one user session and includes demographic, navigational, and transactional attributes. The goal is to predict whether a user made a purchase (binary classification) and to segment users into behavioral clusters.

Variable	Type	Range / Values	Description
user_id	Numerical	1 – 8000	Unique identifier for each user
age	Numerical	18 – 59 years	Age of the user
gender	Categorical	Male / Female	Gender of the user
device_type	Categorical	Mobile / Desktop / Tablet	Device used to access the platform
time_on_site	Numerical	1 – 30 minutes	Total time spent on the website
pages_viewed	Numerical	1 – 19 pages	Number of pages viewed during session
previous_purchases	Numerical	0 – 14	Number of past purchases made by user
cart_items	Numerical	0 – 9 items	Number of items added to the cart
discount_seen	Binary	0 = No / 1 = Yes	whether a discount was shown
ad_clicked	Binary	0 = No / 1 = Yes	whether the user clicked an advertisement
returning_user	Binary	0 = New / 1 = Returning	whether the user is a returning customer

avg_session_time	Numerical	0.57 – 44.75 minutes	Average time spent per session
bounce_rate	Numerical	0.01% – 99.97%	Percentage of sessions with no interaction
purchase	Binary	0 = No / 1 = Yes	whether a purchase was made

3.Problem Statement:

3.1 Business Problem:

In the highly competitive e-commerce industry, organizations face several challenges related to customer engagement and conversion performance. One of the main business problems addressed in this project are:

- Low conversion rate
- Users browsing but not buying
- Inefficient advertising campaigns and marketing strategies
- Poor customer engagement
- lack of personalization
- limited understanding of customer behavior

3.2 Analytical Objectives:

This project aims to address these challenges by:

- predicting whether a user will make a purchase based on their demographic characteristics and browsing behavior.
- understanding customer behavior patterns
- identifying the most important factors influencing purchase decisions
- segmenting customers based on their behavioral characteristics
- improving marketing strategies

Achieving these objectives will enable the organization to better target potential customers, enhance engagement, and optimize overall business performance.

3.3 Main Research Questions:

- What drives a user to make a purchase?

- Which users are at risk of not buying?
- Do users who see discounts buy more?
- What behavioral patterns differentiate buyers from non-buyers?
- Are demographic features important?
- Do discounts and promotions influence purchases?
- How well do models perform on minority (No Buy) users?
- Which model is best for each business objective?

3.4 Target Variable:

Variable name: purchase

Type: Binary

Value 0: The user did not make a purchase during their session

Value 1: The user completed a purchase

It serves as the dependent variable for predicting customer buying behavior based on demographic and browsing features. Analyzing the factors influencing this variable helps improve conversion rates, optimize marketing strategies, and better understand customer decision-making patterns.

4. Data Preparation:

This section describes every preprocessing step applied to the e-commerce dataset. Each decision is justified and the exact Python code is provided matching our project notebook.

4.1 Import Libraries:

All required libraries are imported at the top of the notebook:

Library	Purpose
Pandas	Data manipulation and analysis (loading datasets, cleaning data, handling missing values, and performing transformations using DataFrames)
NumPy	Numerical computations (working with arrays and mathematical operations efficiently)
Matplotlib	Create charts and graphs for data visualization.
Seaborn	Provide advanced visualizations (built on top of Matplotlib)
Scikit-learn	Machine learning models

SMOTE	Handle imbalanced data by generating synthetic samples.
Train-Test Split	Splits the dataset into training and testing sets to evaluate the performance of purchase prediction models.
StandardScaler	Normalizes numerical data by scaling values.
LabelEncoder	Converts categorical values into numeric format.

```
# _____ Import Libraries _____
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Handling class imbalance
from imblearn.over_sampling import SMOTE
```

4.2 Loading & Initial Inspection:

The dataset is loaded with pandas read_csv. Three quick diagnostics are run immediately:

- Shape (8000, 14): 8,000 sessions, 14 attributes
- First five rows (head)
- 160 nulls per column = 2 % missingness

```
# _____ Load dataset _____
df = pd.read_csv(r"C:\Users\ramez\Downloads\e-commerce_user_behavior_8000.csv")

# Display dataset
print(f"Shape: {df.shape}")
print(df.head())

# Check Missing Values
print(df.isnull().sum())
```

4.3 Remove Duplicates:

- `drop_duplicates()` removes any fully identical rows.
- `reset_index(drop=True)` re-sequences the index so downstream integer-based operations remain consistent.

```
# _____ Remove duplicates _____  
df = df.drop_duplicates().reset_index(drop=True)
```

0 duplicates found. Dataset: 8,000 rows × 14 columns.

4.4 Drop Identifier Column (user_id):

`user_id` is a surrogate key with no predictive or analytical value. Including it in model training would cause data leakage because the model could 'memorise' individual IDs rather than learning generalizable patterns.

- Identifier columns always get dropped before any ML step

```
# _____ Drop identifier _____  
df = df.drop(columns=['user_id'])
```

user_id dropped. Dataset: 8,000 rows × 13 columns.

4.5 Handling Missing Values:

Rather than dropping rows, which would remove 24.8% of the data and risk losing all 13 rare non-buyers, we apply statistical imputation:

- Numeric columns are filled with their column median (robust to skew and outliers).
- Categorical columns are filled with the column mode (most frequent category).

Imputation is done before train/test split to avoid fitting on test data leaking into the imputer.

```
# _____ Handle Missing Values _____  
  
# Fill numeric values with median  
num_cols = df.select_dtypes(include=np.number).columns  
df[num_cols] = df[num_cols].fillna(df[num_cols].median())  
  
# Fill categorical values with mode  
cat_cols = df.select_dtypes(include='object').columns  
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
```


Feature	Method Used	Replacement Value
Age	Median	39.0 years
Time on Site	Median	15.67 minutes
Pages Viewed	Median	10.0 pages
Previous Purchases	Median	7.0
Cart Items	Median	4.0 items
Average Session Time	Median	14.98 minutes
Bounce Rate	Median	49.53%
Discount Seen	Median	1 (Yes)
Ad Clicked	Median	1 (Yes)
Returning User	Median	1 (Yes)
Gender	Mode	Male
Device Type	Mode	Mobile

4.6 Outlier Detection (IQR Method):

For each of the seven continuous features the Interquartile Range (IQR) fence is computed:

- lower = $Q1 - 1.5 \times IQR$,
- upper = $Q3 + 1.5 \times IQR$.

Any value outside this range is flagged as a potential outlier.

The scan found 8 outliers only in avg_session_time (only 0.1 % of 8,000 rows), all other features returned zero outliers.

- IQR fencing is non-parametric and works well even when features are not normally distributed

```
# _____ Continuous features to check for outliers _____
num_cols = ['age', 'time_on_site', 'pages_viewed',
            'previous_purchases', 'cart_items',
            'avg_session_time', 'bounce_rate']

for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    n_out = ((df[col] < lower) | (df[col] > upper)).sum()
    print(f'{col}: {n_out} outliers')
```

4.7 Encoding Categorical Variables:

Two encoding strategies are applied depending on the nature of each categorical feature:

Feature	Encoding Technique	Transformed Variables
Gender	Label Encoding	Female → 0, Male → 1
Device Type	One-Hot Encoding	device_Desktop, device_Mobile, device_Tablet (Binary values: 0 or 1)

```
# _____ Encoding _____
le = LabelEncoder()
df['gender'] = le.fit_transform(df['gender'])

df = pd.get_dummies(df, columns=['device_type'], prefix='device')

# _____ Convert bool to int _____
bool_cols = df.select_dtypes(include='bool').columns
df[bool_cols] = df[bool_cols].astype(int)
```

After encoding: dataset grows from 13 to 15 columns (14 features + 1 target).
Shape: (8,000 × 15).

4.8 Save Cleaned Dataset:

The fully preprocessed dataframe is persisted to disk as a CSV. This checkpoint enables reproducibility and allows the cleaned data to be shared or reloaded without repeating earlier steps.

- `index=False` prevents pandas from writing the row index as an extra column
- Saved at this point, before SMOTE and scaling, so the file represents the true cleaned distribution.

```
# _____ Save cleaned dataset _____  
df.to_csv("ecommerce_user_behavior_cleaned.csv", index=False)  
print("Cleaned dataset saved successfully!")
```

4.9 Separate Features & Target:

The feature matrix X is formed by dropping the target column `purchase`. The target vector y is extracted and cast to integer type (0 = no purchase, 1 = purchase). Inspecting the value counts reveals a severe class imbalance (7,987 purchases vs only 13 non-purchases) motivating SMOTE oversampling.

```
# _____ Separate features and target _____  
X = df.drop(columns=['purchase'])  
y = df['purchase'].astype(int)  
  
print("Before balancing:")  
print(y.value_counts())
```

4.10 Train / Test Split:

The dataset is divided into 80 % training and 20 % test sets with `stratify=y` to preserve the class ratio in both sets.

- `random_state=42` ensures reproducibility across runs
- `stratify=y` is critical here: without it, the 13 minority samples could all fall into training or all into test, biasing evaluation
- SMOTE is applied ONLY to the training set after this split to avoid data leakage.

```
# _____ Train Test Split _____
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

4.11 Class Imbalance — SMOTE (Partial Balancing):

SMOTE (Synthetic Minority Over-sampling Technique) generates synthetic samples for the minority class by interpolating between existing minority samples in feature space.

- `sampling_strategy=0.4` targets a 40 % minority ratio rather than full 50/50 balance, which retains some of the natural imbalance signal while giving the model enough minority examples to learn from.
- After SMOTE: 6,390 majority vs 2,556 minority.
- SMOTE is applied to `X_train` only to prevent synthetic samples from leaking into the evaluation set.
- Partial balancing (40 %) is often preferable to full balancing when the true population is also imbalanced, avoiding over-correction.
- SMOTE requires all features to be numeric, the reason why encoding was performed beforehand

```
# _____ SMOTE (Partial Balancing) _____
smote = SMOTE(
    sampling_strategy=0.4, # 40% balance (not equal)
    random_state=42
)

X_train_bal, y_train_bal = smote.fit_resample(X_train, y_train)

print("After SMOTE:")
print(y_train_bal.value_counts())
```

Stage	Buyers (Class 1)	Non-Buyers (Class 0)	Total Observations	Minority Percentage
-------	---------------------	-------------------------	-----------------------	------------------------

Before SMOTE	6,390	10	6,400	0.16%
After SMOTE	6,390	2,556	8,946	28.6%

4.12 Feature Scaling– StandardScaler :

Continuous features are standardized to zero mean and unit variance using StandardScaler.

- The scaler is fit only on the SMOTE-augmented training set then applied (transform only) to the test set preventing test-set statistics from influencing the scaler parameters.
- Scaling is critical for KNN and Logistic Regression (distance / gradient-based), but beneficial even for tree-based models in practice

```
# _____ Feature Scaling _____
scale_cols = [
    'age', 'time_on_site', 'pages_viewed',
    'previous_purchases', 'cart_items',
    'avg_session_time', 'bounce_rate'
]

scaler = StandardScaler()

X_train_bal[scale_cols] = scaler.fit_transform(X_train_bal[scale_cols])
X_test[scale_cols] = scaler.transform(X_test[scale_cols])
```

Binary features (gender, discount_seen, ad_clicked, returning_user, device *) are NOT scaled — they already carry values of 0 or 1

4.13 Full Pipeline Summary:

All preprocessing steps and their outcomes are summarised below. The four final variables feed directly into all models in Part 3.

Step	Process	Key Parameter / Output
2	Loading & Initial Inspection	8,000 × 14 dataset, 160 NaN per column, 13 non-buyers
3	Remove Duplicates	0 duplicates removed, 8,000 rows retained

4	Drop Identifier Column (user_id)	Removed — no predictive value → 13 columns remaining
5	Missing Value Imputation	Median (numeric) + Mode (categorical) → 0 missing values, all 8,000 rows retained
6	Outlier Detection	8 outliers detected in avg_session_time → retained (tree models robust)
7	Encoding Categorical Variables	gender → Label Encoding (Female = 0, Male = 1) ; device_type → One-Hot Encoding (device_Desktop, device_Mobile, device_Tablet) → 15 columns
8	Save Cleaned Dataset	ecommerce_user_behavior_cleaned.csv · 8,000 × 15
9	Train / Test Split	80/20 stratified · Train: 6,400 · Test: 1,600
10	SMOTE (strategy = 0.4)	Training set expanded to 8,946 rows · minority: 10 → 2,556 (+2,546 synthetic)
11	StandardScaler	7 continuous features normalized · fitted on training data only

Dataset	Dimensions	Description
X_train_bal	(8,946 × 14)	Training features after SMOTE balancing and scaling
y_train_bal	(8,946,)	Balanced target variable including synthetic minority samples (6,390 buy + 2,556 no-buy / 2,546 synthetic)
X_test	(1,600 × 14)	Test features scaled using training parameters (real distribution untouched)
y_test	(1,600,)	Original test labels without modification (1,597 buy + 3 no-buy)

4.14 Features Summary:

After all steps the training set contains 8,946 rows (6,390 majority + 2,556 SMOTE-generated minority) with 16 fully numeric, scaled features, ready for Logistic Regression, Decision Tree and Random Forest:

Column	Type	Description
age	Continuous	User age in years
gender	Binary (0/1)	Encoded: Female = 0, Male = 1
time_on_site	Continuous	Total session time in minutes
pages_viewed	Discrete	Count of pages visited
previous_purchases	Discrete	Historical purchase count
cart_items	Discrete	Items added to cart this session
discount_seen	Binary	1 if a discount promotion was shown
ad_clicked	Binary	1 if user clicked an advertisement
returning_user	Binary	1 if repeat/returning visitor
avg_session_time	Continuous	Average time per page (minutes)
bounce_rate	Continuous	Percent of single-page sessions (%)
device_Desktop	OHE Binary	1 if session was on a desktop device
device_Mobile	OHE Binary	1 if session was on a mobile device
device_Tablet	OHE Binary	1 if session was on a tablet device
purchase	Target	1 = purchase made, 0 = no purchase

5. Methods And Analysis:

5.1 Overview:

This part provides a complete, step-by-step explanation of the Data Mining section that covers three classification models, a dimensionality reduction technique (PCA), and a comprehensive multi-model evaluation framework.

The mining phase has the following high-level structure:

- Logistic Regression: linear baseline model with coefficient analysis
- Decision Tree: rule-based model with tree visualisation and feature importance
- Random Forest: ensemble model with aggregated feature importance
- PCA — 3D projection for exploratory visual analysis
- Model Evaluation: per-model metrics and side-by-side confusion matrices.

NB: Warnings Library:

```
import warnings
warnings.filterwarnings('ignore')
```

Used to suppress unnecessary warnings and keep notebook output clean during hyperparameter tuning.

5.2 Logistic Regression

→ Why Logistic Regression?

Logistic Regression is the first and simplest model applied. It serves as a linear baseline, allowing the team to understand which features most strongly push predictions in either direction (purchase or no purchase).

5.2.1 Model Library:

```
# _____ Models _____
from sklearn.linear_model import LogisticRegression
```

5.2.2 Model Instantiation and Training:

```
# — Logistic Regression —
lr = LogisticRegression(class_weight=None, max_iter=1000, random_state=42)
lr.fit(X_train_bal, y_train_bal)
```


Line-by-line breakdown:

Parameter / Call	Purpose
<code>class_weight = None</code>	No internal class re-weighting is applied. Class imbalance was already handled upstream by SMOTE, so this is intentionally left as None.
<code>max_iter = 1000</code>	Allows the solver up to 1,000 optimization iterations to converge. The default of 100 is often insufficient for complex datasets.
<code>random_state = 42</code>	Fixes the random seed so results are perfectly reproducible across runs.
<code>lr.fit(X_train_bal, y_train_bal)</code>	Trains the model on the SMOTE-balanced training data. The model learns the weight (coefficient) of each feature that maximizes the log-likelihood of the training labels

5.2.3 Generating Predictions:

```
y_pred_lr = lr.predict(X_test)
```

Applies the trained model to the held-out test set (20% of original data). Each row is assigned either 0 (No Buy) or 1 (Buy) based on whether the predicted probability exceeds 0.5. Predictions are stored in `y_pred_lr` for later evaluation.

5.2.4 Feature Coefficient Extraction:

```
# Feature coefficients – direction and strength of each variable
coef_df = pd.DataFrame({'Feature': X_train_bal.columns, 'Coefficient': lr.coef_[0]})
coef_df = coef_df.sort_values('Coefficient', ascending=False)
print(coef_df)
```

What this does:

- `lr.coef_[0]` accesses the array of learned weights, one per feature.
- A positive coefficient means the feature increases the probability of a purchase.
- A negative coefficient means the feature decreases the probability of a purchase.

- Sorting in descending order makes the most influential positive features appear at the top of the table.
- This output acts as an interpretability report, showing which variables the model relies on most heavily.

5.2.5 Coefficient Visualisation:

```
# Visualise coefficients
plt.figure(figsize=(9, 5))
sns.barplot(data=coef_df, x='Coefficient', y='Feature', palette='coolwarm')
plt.title('Logistic Regression – Feature Coefficients')
plt.axvline(x=0, color='black', linewidth=0.8, linestyle='--')
plt.tight_layout()
plt.savefig('lr_coefficients.png', dpi=150)
plt.show()
```

Line-by-line breakdown:

Line	Purpose
<code>figsize = (9, 5)</code>	Sets chart dimensions to 9 × 5 inches, wide enough to display long feature names without truncation.
<code>sns.barplot(data=coef_df, x='Coefficient', y='Feature', palette='coolwarm')</code> <code>y='Feature', palette='coolwarm')</code>	Uses a diverging red-to-blue color scheme; positive values appear in warm tones, negative values in cool tones
<code>plt.axvline(x = 0, color='black', linewidth=0.8, linestyle='--')</code> <code>plt.tight_layout()</code>	Draws a vertical reference line at zero to clearly separate positive and negative feature impacts.
<code>plt.savefig('lr_coefficients.png', dpi = 150)</code>	Saves the chart as a high-resolution PNG (150 DPI), suitable for reports or presentations

5.3 Decision Tree

→ Why Decision Tree?

The Decision Tree model creates an explicit, human-readable set of if-then rules. Unlike Logistic Regression, it captures non-linear relationships and interactions between features, making it highly interpretable for business stakeholders.

5.3.1 Model Library:

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

5.3.2 Model Instantiation and Training:

```
# — Decision Tree —————  
dt = DecisionTreeClassifier(max_depth=5, class_weight='balanced', random_state=42)  
dt.fit(X_train_bal, y_train_bal)  
y_pred_dt = dt.predict(X_test)
```

Line-by-line breakdown:

Parameter / Step	Purpose
max_depth=5	Limits the decision tree to 5 levels of splits to prevent overfitting while still capturing meaningful patterns.
class_weight='balanced'	Adjusts class weights inversely to their frequency, helping reduce class imbalance impact (complements SMOTE).
random_state=42	Ensures reproducible results by fixing randomness during split decisions.
dt.fit(X_train_bal, y_train_bal)	Trains the decision tree by recursively splitting data using the feature and threshold that best reduces impurity (e.g., Gini).
y_pred_dt = dt.predict(X_test)	Uses the trained tree to classify test samples by following decision paths to leaf nodes.

5.3.3 Tree Structure Visualisation:

```
# Visualise the tree structure (top 3 levels for readability)
plt.figure(figsize=(20, 8))
plot_tree(dt, max_depth=3, feature_names=X_train_bal.columns,
          class_names=['No Buy', 'Buy'], filled=True, rounded=True)
plt.title('Decision Tree – Purchase Decision Rules (Top 3 Levels)', fontsize=13)
plt.tight_layout()
plt.savefig('decision_tree.png', dpi=150, bbox_inches='tight')
plt.show()
```

Detailed explanation:

Parameter / Step	Purpose
figsize=(20, 8)	Sets a wide figure size to accommodate the horizontally expanding tree and prevent overlapping node labels.
max_depth=3	Displays only the first 3 levels of the trained tree to keep the visualization readable while highlighting the most important decision rules.
feature_names=X_train_bal.columns	Replaces numeric feature indices with actual column names, making each split interpretable.
class_names=['No Buy', 'Buy']	Assigns human-readable labels to the predicted classes at the leaf nodes.
filled=True	Colors nodes based on the dominant class to improve visual interpretation of decision patterns.
rounded=True	Uses rounded corners for nodes to enhance visual clarity and aesthetics.
bbox_inches='tight'	Ensures the saved image is fully captured without cropping important parts at the edges.

5.3.4 Decision Tree Feature Importance:

```
# Feature importance bar chart
fi_dt = pd.DataFrame({'Feature': X_train_bal.columns, 'Importance': dt.feature_importances_})
fi_dt = fi_dt.sort_values('Importance', ascending=False)
plt.figure(figsize=(9, 5))
sns.barplot(data=fi_dt, x='Importance', y='Feature', palette='Blues_r')
plt.title('Decision Tree – Feature Importance')
plt.tight_layout()
plt.savefig('dt_feature_importance.png', dpi=150)
plt.show()
```

Key concepts:

- `dt.feature_importances_` returns the mean decrease in Gini impurity attributable to each feature across all nodes where that feature is used to split.
- Features used higher in the tree (closer to the root) tend to have higher importance because they affect more samples.
- Sorting descending and using `palette='Blues_r'` (reversed blues) places the most important feature at the top with the darkest shade.
- This bar chart is a quick diagnostic: if only one or two features dominate, the model may be overly reliant on them and could be fragile.

5.4 Random Forest

→ Why Random Forest?

Random Forest is an ensemble method that trains many independent decision trees and aggregates their predictions by majority vote. It is generally the strongest of the three models, more robust to noise and overfitting than a single Decision Tree, while still providing feature importance estimates.

5.4.1 Model Library:

```
from sklearn.ensemble import RandomForestClassifier
```

5.4.2 Model Instantiation and Training:

```
# — Random Forest —
rf = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    class_weight=None,
    random_state=42,
    n_jobs=-1
)
rf.fit(X_train_bal, y_train_bal)
y_pred_rf = rf.predict(X_test)
```

Line-by-line breakdown:

Parameter / Step	Purpose
n_estimators=200	Builds 200 decision trees; increasing the number of trees improves model stability and reduces variance while balancing computation cost.
max_depth=10	Allows each tree to grow up to 10 levels. Deeper trees are acceptable because ensemble averaging reduces overfitting risk.
class_weight=None	No manual class weighting is applied since class imbalance is already handled using SMOTE.
random_state=42	Ensures reproducibility by fixing randomness in bootstrap sampling and feature selection at each split.
n_jobs=-1	Utilizes all available CPU cores to parallelize training and speed up computation.
rf.fit(...)	Trains each tree on a bootstrap sample of the training data while randomly selecting features at each split.
y_pred_rf = rf.predict(X_test)	Aggregates predictions from all trees; the final class is determined by majority voting across the ensemble.

5.4.3 Random Forest Feature Importance:

```
# Feature importance – averaged across all 200 trees
fi_rf = pd.DataFrame({'Feature': X_train_bal.columns, 'Importance': rf.feature_importances_})
fi_rf = fi_rf.sort_values('Importance', ascending=False)
plt.figure(figsize=(9, 5))
sns.barplot(data=fi_rf, x='Importance', y='Feature', palette='Blues_r')
plt.title('Random Forest – Feature Importance (200 Trees)')
plt.tight_layout()
plt.savefig('rf_feature_importance.png', dpi=150)
plt.show()
```

Key differences from the Decision Tree importance:

- `rf.feature_importances_` averages the Gini-based importance across all 200 trees, making it far more stable and reliable than a single-tree estimate.
- Random feature selection at each split means every feature gets a fair chance to be evaluated, reducing the tendency for one or two dominant features to overshadow others.
- Comparing this chart with the Decision Tree's importance chart reveals whether feature rankings are consistent across model types, a strong sign of genuine predictive signal.

5.5 PCA — 3D Feature Space Projection:

→ Why PCA?

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique. It is used to reduce the multi-dimensional feature space down to 3 dimensions so that the spatial separation between purchasing and non-purchasing customers can be observed in a 3D scatter plot.

5.5.1 PCA Library:

```
from sklearn.decomposition import PCA
```

5.5.2 Required Import :

```
from mpl_toolkits.mplot3d import Axes3D
```

Although PCA itself only requires `sklearn.decomposition`, the 3D scatter plot requires a 3D axes object to switch matplotlib from its default 2D canvas to a full 3D

coordinate system with depth, rotation, and perspective. This import brings in the Axes3D class from matplotlib's 3D toolkit.

5.5.3 Fitting PCA — Reducing to 3 Components :

```
pca3 = PCA(n_components=3, random_state=42)
X_pca3 = pca3.fit_transform(X_train_bal)
```

Line-by-Line Explanation:

Code	Explanation
<pre>pca3 = PCA(n_comp onents=3, random_sta te=42)</pre>	Initializes a PCA model using scikit-learn. • n_components = 3 → Reduces the dataset to 3 principal components (X, Y, Z axes). • random_state = 42 → Ensures reproducibility by fixing the random seed for consistent results across runs.
<pre>X_pca3 = pca3.fit_t ransform(X _train_bal)</pre>	Applies PCA in two steps: fit + transform: Fit phase: centers the data, computes covariance, performs SVD, and extracts eigenvectors (principal directions) and eigenvalues (variance), selecting the top 3 components. Transform phase: projects data onto the 3 principal components, producing <code>X_pca3</code>, a NumPy array where each row represents a data point in reduced 3D space.

❖ Why use `X_train_bal` and not `X_test`?

PCA is fitted only on the training set to prevent data leakage. The training set here is the SMOTE-balanced version (`X_train_bal`), which ensures the PCA axes reflect the distribution used for model training, not future/unseen data.

5.5.4 Printing Explained Variance:


```

explained_var = pca3.explained_variance_ratio_
print(f"PC1: {explained_var[0]:.4f}")
print(f"PC2: {explained_var[1]:.4f}")
print(f"PC3: {explained_var[2]:.4f}")
print(f"Total Explained: {explained_var.sum():.4f}")

```

Line-by-Line Explanation:

Code	Explanation
<pre>explained_var = pca3.explained_variance_ratio_</pre>	Retrieves the explained variance ratio for each principal component. Each value represents the proportion of total dataset variance captured by that component.
<pre>print(f"PC1: {explained_var[0]:.4f}") print(f"PC2: {explained_var[1]:.4f}") print(f"PC3: {explained_var[2]:.4f}")</pre>	Displays the explained variance of PC1, PC2, and PC3. Values are formatted to 4 decimal places (: .4f). PC1 typically has the highest value since PCA ranks components by descending variance.
<pre>print(f"Total Explained: {explained_var.sum():.4f}")</pre>	Computes the total variance explained by the selected components by summing their ratios. A value close to 1.0 indicates most information is preserved, while lower values indicate significant information loss.

5.5.5 Building the PCA DataFrame:

```
# DataFrame
pca3_df = pd.DataFrame({
    'PC1': X_pca3[:, 0],
    'PC2': X_pca3[:, 1],
    'PC3': X_pca3[:, 2],
    'Purchase': y_train_bal.values
})
```

This block converts the raw NumPy array `X_pca3` into a pandas DataFrame for easier downstream use. Each argument constructs one column:

Column	Description
PC1	First principal component (most significant), extracted using <code>X_pca3[:, 0]</code> by selecting all rows from column index 0 of the transformed dataset.
PC2	Second principal component, extracted using <code>X_pca3[:, 1]</code> by selecting all rows from column index 1.
PC3	Third principal component, extracted using <code>X_pca3[:, 2]</code> by selecting all rows from column index 2.
Purchase	Binary target variable representing customer behavior. The <code>.values</code> method converts it to a NumPy array to ensure proper alignment and avoid indexing issues during analysis.

5.5.6 Sampling for Plot Clarity:

```
# Sample to avoid clutter
pca3_sample = pca3_df.sample(3000, random_state=42)
```

- `.sample(3000, random_state=42)` randomly selects 3,000 rows from `pca3_df` without replacement.
- The `random_state=42` parameter ensures reproducibility, so the same subset is used in every run.
- The choice of 3,000 points provides a balance between maintaining a representative distribution and ensuring efficient, readable visualization in Matplotlib.

5.5.7 Creating the 3D Scatter Plot:

```
# Plot 3D
fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    pca3_sample['PC1'],
    pca3_sample['PC2'],
    pca3_sample['PC3'],
    c=pca3_sample['Purchase'],
    cmap='coolwarm',
    alpha=0.6
)
```

Line-by-Line Explanation:

Code / Step	Explanation
<code>fig = plt.figure(figsize=(9, 7))</code>	Creates the main Matplotlib figure (canvas) with dimensions 9×7 inches, required before adding any plots.
<code>ax = fig.add_subplot(111, projection='3d')</code>	Adds a single 3D subplot (1 row, 1 column, position 1). The <code>projection='3d'</code> enables 3D axes and returns an <code>ax</code> object for 3D plotting.
<code>scatter = ax.scatter(...)</code>	Generates a 3D scatter plot by plotting data points in three-dimensional space, with parameters controlling visual properties such as position, color, and appearance.

Argument	Description
----------	-------------

<code>pca3_sample['PC1']</code>	X-axis values representing the first principal component.
<code>pca3_sample['PC2']</code>	Y-axis values representing the second principal component.
<code>pca3_sample['PC3']</code>	Z-axis values representing the third principal component.
<code>c = pca3_sample['Purchase']</code>	Color encoding based on the target variable (0 = No Purchase, 1 = Purchase) to distinguish classes.
<code>cmap = 'coolwarm'</code>	Diverging colormap: blue for non-buyers and red for buyers, improving class separation.
<code>alpha = 0.6</code>	Sets point transparency to 60%, reducing clutter and improving visibility of overlapping points.

5.5.8 Setting Axis Labels, Saving and Displaying the 3D Plot:

```
ax.set_title('3D PCA – Customer Behavior')
ax.set_xlabel(f'PC1 ({explained_var[0]*100:.1f}%)')
ax.set_ylabel(f'PC2 ({explained_var[1]*100:.1f}%)')
ax.set_zlabel(f'PC3 ({explained_var[2]*100:.1f}%)')

plt.tight_layout()
plt.savefig('pca_3d.png', dpi=150)
plt.show()
```

- `ax.set_xlabel`, `ax.set_ylabel`, `ax.set_zlabel` label the three axes (PC1, PC2, PC3) including the percentage of variance explained by each component.
- `plt.tight_layout()` adjusts spacing ensuring that all plot elements are fully visible..
- `plt.savefig('pca_3d.png', dpi=150)` saves the figure as a PNG file with sufficient resolution for clear use.
- `plt.show()` displays the plot for visualization.

5.5.9 PCA Feature Loadings:

```
loadings = pd.DataFrame(  
    pca3.components_.T,  
    columns=['PC1', 'PC2', 'PC3'],  
    index=X_train_bal.columns  
)  
  
# Focus on PC1 (most important)  
pc1_sorted = loadings['PC1'].abs().sort_values(ascending=False)  
  
print("\nTop features influencing behavior (PC1):")  
print(pc1_sorted.head(10))
```

Code	Explanation
<code>pca3.components_.T</code>	Retrieves the PCA loading matrix by transposing the component matrix. It converts the shape from (3, <i>n_features</i>) to (<i>n_features</i> , 3) so each row represents a feature and each column a principal component.
<code>pd.DataFrame(..., columns=['PC1', 'PC2', 'PC3'], index=X_train_bal.columns)</code>	Converts the loading matrix into a readable DataFrame where rows are original features and columns are principal components. Each value shows a feature's contribution to a given component. .
<code>loadings['PC1'].abs().sort_values(ascending=False)</code>	Ranks features by their contribution to PC1 using absolute values <code>.abs()</code> , ignoring direction. Higher magnitude indicates stronger importance while sign indicates direction.
<code>print(pc1_sorted.head(10))</code>	Displays the top 10 most influential features for PC1 using <code>head(10)</code> , highlighting the main drivers of variance in the dataset.

5.6 Per-Model Evaluation:

After all three models have made predictions on the test set, this part computes and prints a complete performance report for each model, covering four complementary metrics.

5.6.1 Per-Model Libraries:

```
# _____ Evaluation _____  
from sklearn.metrics import (  
    classification_report,  
    confusion_matrix,  
    accuracy_score,  
    f1_score,  
    roc_auc_score  
)
```

- **Accuracy Score** → Overall prediction accuracy
- **F1 Score** → Balance between precision and recall
- **Confusion Matrix** → Model prediction errors
- **Classification Report** → Full performance summary
- **roc_auc_score** → evaluation of a model's distinguishment level between classes

5.6.2 Model and Estimator Dictionaries:

```
# Dictionary of prediction arrays (for metrics)  
models = {  
    'Logistic Regression': y_pred_lr,  
    'Decision Tree':      y_pred_dt,  
    'Random Forest':      y_pred_rf,  
}  
  
# Dictionary of fitted estimators (needed for predict_proba / ROC-AUC)  
fitted = {  
    'Logistic Regression': lr,  
    'Decision Tree':      dt,  
    'Random Forest':      rf,  
}
```

Two separate dictionaries are maintained:

- **models:** maps each model name to its hard predictions (0 or 1 arrays), used for Accuracy, F1, classification report, and confusion matrix, all of which require discrete class labels.
- **fitted:** maps each model name to its trained estimator object, needed to call `.predict_proba()` for the ROC-AUC score, which requires probability estimates rather than hard predictions.

5.6.3 Metric Computation Loop:

```
for name, pred in models.items():
    print(f'\n=== {name} ===')
    print(f'Accuracy : {accuracy_score(y_test, pred):.4f}')
    print(f'F1 Score : {f1_score(y_test, pred, average="weighted"): .4f}')
    y_prob = fitted[name].predict_proba(X_test)[: , 1]
    print(f'ROC-AUC : {roc_auc_score(y_test, y_prob):.4f}')
    print(classification_report(y_test, pred,
                                target_names=[ 'No Buy', 'Buy']))
```

Metric-by-metric breakdown:

Metric	What It Measures
<code>accuracy_score</code>	Proportion of correctly classified predictions (both classes). Ranges from 0 to 1, but can be misleading in imbalanced datasets since predicting the majority class can still yield high accuracy.
<code>f1_score</code> (<code>average='weighted'</code>)	Weighted harmonic mean of precision and recall across classes. Better than accuracy for imbalanced data because it balances false positives and false negatives while accounting for class distribution.
<code>predict_proba(X_test)[: , 1]</code>	Extracts the predicted probability of the positive class ("Buy") for each sample from the model's probability output.

roc_auc_score	Measures how well the model ranks positive cases above negative ones across all thresholds. Ranges from 0.5 (random performance) to 1.0 (perfect classification), making it threshold-independent.
classification_report	Provides detailed per-class evaluation including precision, recall, and F1-score, along with macro and weighted averages for an overall performance view.

5.7 Confusion Matrices — All Models:

The final block produces a side-by-side visualisation of the confusion matrices for all three models, making it easy to compare how each model makes its mistakes.

5.7.1 Metric Computation Loop:

```
# — Confusion matrices – all 3 models side by side —
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
```

`plt.subplots(1, 3)` creates a single figure with three subplots arranged in one row and three columns. `axes` is an array of three Matplotlib Axes objects, one per model. `figsize=(14, 4)` is deliberately wide to accommodate three side-by-side heatmaps.

5.7.2 Confusion Matrix Loop:

```
for ax, (name, pred) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt='d', ax=ax, cmap='Blues', cbar=False,
                xticklabels=['No Buy', 'Buy'],
                yticklabels=['No Buy', 'Buy'])
    ax.set_title(name, fontsize=11)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
```

Line-by-line breakdown:

Parameter / Call	Purpose
------------------	---------

<code>zip(axes, models.items())</code>	Matches each subplot (Axes) with a model and its predictions, ensuring each confusion matrix is plotted in the correct panel.
<code>confusion_matrix(y_test, pred)</code>	Produces a 2×2 matrix showing TN, FP, FN, and TP, summarizing correct and incorrect classifications.
<code>sns.heatmap(..., annot=True, fmt='d')</code>	Displays the confusion matrix as a heatmap with exact integer values shown in each cell (no scientific notation).
<code>cmap='Blues'</code>	Applies a blue gradient where darker shades represent higher values, helping highlight correct predictions on the diagonal.
<code>cbar=False</code>	Removes the color bar since values are already explicitly shown and multiple subplots share the same structure.
<code>xticklabels / yticklabels</code>	Replaces numeric labels (0, 1) with meaningful class names ("No Buy", "Buy") for easier interpretation.

5.7.3 Final Layout and Save:

```
plt.suptitle('Confusion Matrices – All Models', fontsize=13, y=1.02)
plt.tight_layout()
plt.savefig('confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
```

Key concepts:

- `plt.suptitle()` places a single centred title above all three subplots. `y=1.0` nudges it slightly above the top edge so it does not overlap the individual subplot titles.
- `plt.tight_layout()` automatically adjusts spacing between subplots to eliminate overlapping labels.
- `bbox_inches='tight'` ensures the super-title is included within the saved image boundary and is not cropped.

5.8 Summary of The Mining Phase:

The table below summarises all major steps in the mining section, their corresponding code components, and their analytical purpose.

5	Component	Purpose
1	Logistic Regression	Linear baseline model; coefficient analysis shows directional influence of features on purchase probability.
2	LR Coefficient Chart	Diverging bar chart (coolwarm) with zero reference line; highlights features that increase vs. decrease purchase likelihood.
3	Decision Tree	Rule-based non-linear classifier; <code>max_depth=5</code> limits overfitting; <code>class_weight='balanced'</code> handles class imbalance.
4	Tree Visualisation	Simplified top-3-level tree diagram showing decision rules, Gini impurity, and sample distribution per node.
5	DT Feature Importance	Bar chart representing average Gini impurity reduction contributed by each feature.
6	Random Forest	Ensemble of 200 decision trees; improves stability and accuracy; <code>n_jobs=-1</code> enables parallel training.
7	RF Feature Importance	Aggregated feature importance across all 200 trees, providing a more stable ranking than a single tree.
8	PCA Projection	Unsupervised dimensionality reduction projecting test data into 3D for exploratory visualization.
9	Evaluation Loop	Computes Accuracy, Weighted F1-score, and ROC-AUC for each model; <code>classification_report</code> adds per-class metrics.
10	Confusion Matrices	Side-by-side heatmaps showing TP, TN, FP, and FN for all three models for direct comparison.

6. Results And Interpretations:

6.1 Logistic Regression — Feature Coefficients & Model Performance:

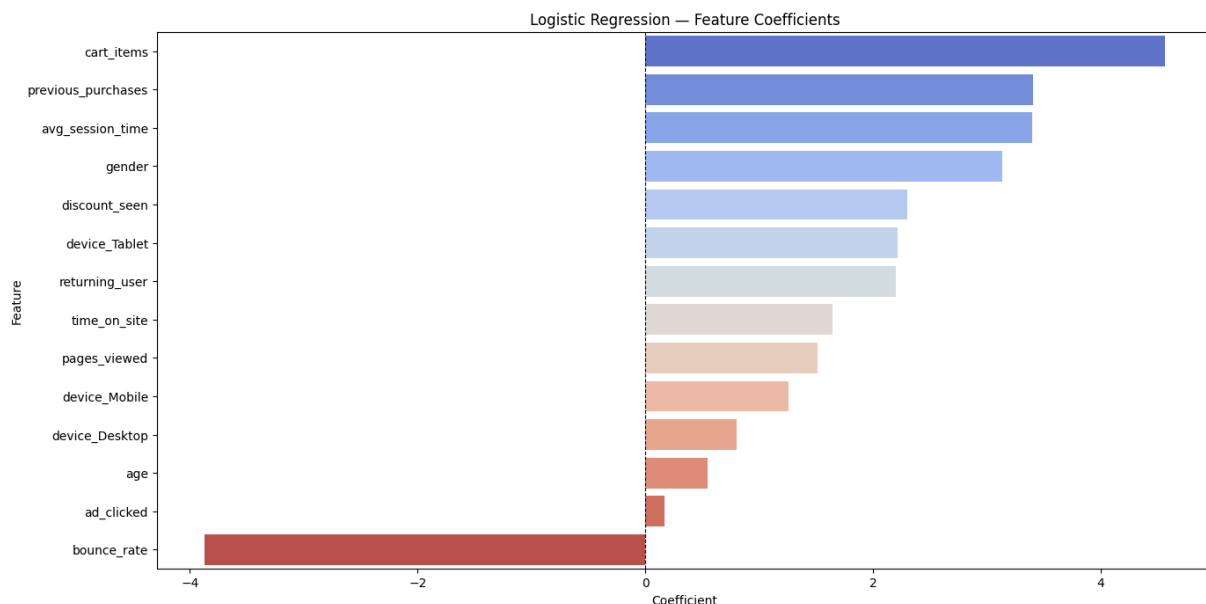
	Feature	Coefficient
5	cart_items	4.516116
9	avg_session_time	3.252491
4	previous_purchases	3.240477
13	device_Tablet	2.846320
1	gender	2.829980
6	discount_seen	2.248706
8	returning_user	2.036088
2	time_on_site	1.688744
3	pages_viewed	1.496851
12	device_Mobile	1.462859
11	device_Desktop	0.996796
0	age	0.514507
7	ad_clicked	0.244735
10	bounce_rate	-3.790834

Coefficient Table Analysis:

Feature	Coefficient	Direction	Interpretation
cart_items	4.516	Strong positive	The number of items in the cart is the most powerful predictor of purchase; each additional item significantly increases purchase likelihood.
avg_session_time	3.252	Positive	Longer average session duration reflects deeper engagement and stronger purchase intent.

previous_purchases	3.240	Positive	Past purchasing behaviour is a strong indicator of customer loyalty and future conversions.
device_Tablet	2.846	Positive	Tablet users are considerably more likely to convert compared to the baseline device category.
gender	2.830	Positive	A notable demographic factor, likely reflecting gender-related shopping patterns.
discount_seen	2.249	Positive	Exposure to discounts meaningfully increases the probability of conversion.
returning_user	2.036	Positive	Returning customers exhibit higher purchase intent than new users.
time_on_site	1.689	Positive	Increased browsing time is associated with a higher likelihood of purchase.
pages_viewed	1.497	Positive	Viewing more pages suggests exploratory behaviour and increased conversion probability.
device_Mobile	1.463	Positive	Mobile users show a moderate increase in likelihood to convert.
device_Desktop	0.997	Positive	Desktop users have a slight positive effect, lower than mobile and tablet users.

age	0.515	Weak positive	Age has a minimal positive influence on purchase behaviour.
ad_clicked	0.245	Weak positive	Clicking on ads has only a limited impact on conversion likelihood.
bounce_rate	-3.791	Strong negative	A high bounce rate strongly reduces purchase probability; users who leave quickly rarely convert.



Key Observations from the Feature Coefficients Bar Chart:

The horizontal bar chart visualises the 14 feature coefficients, sorted by descending absolute magnitude. The blue bars extend rightward (positive effect) and the single red bar (bounce_rate) extends leftward (negative effect). Several patterns emerge:

- The top tier of positive predictors (cart_items, avg_session_time, previous_purchases) all reflect sustained, deliberate customer engagement, users who are actively shopping, not just browsing casually.
- The device_Tablet coefficient (2.846) surpassing device_Mobile (1.463) and device_Desktop (0.997) suggests that tablet users have a distinctly different and more conversion-friendly browsing behaviour possibly due to comfortable screen size for product inspection.

- gender (2.830) is a notably strong predictor, which warrants further investigation: it likely reflects a gender segment with systematically higher purchase rates in this product category.
- bounce_rate (-3.791) stands alone as a strong negative predictor. Its magnitude rivals cart_items, confirming that user disengagement is as informative as engagement for predicting non-purchase.
- ad_clicked (0.245) has a surprisingly small coefficient. This suggests that in this dataset, clicking an ad does not reliably translate into a purchase — a commercially important finding.

```

=== Logistic Regression ===
Accuracy : 0.9969
F1 Score : 0.9974
ROC-AUC : 0.9985

              precision    recall  f1-score   support

   No Buy      0.33        0.67        0.44         3
     Buy      1.00        1.00        1.00    1597

 accuracy              1.00        1600
 macro avg      0.67        0.83        0.72    1600
 weighted avg    1.00        1.00        1.00    1600

```

Model Performance Metrics:

99.69% Accuracy	99.74% F1-Score	99.85% ROC-AUC
---------------------------	---------------------------	--------------------------

Class	Precision	Recall	F1-Score	Support
No Buy (minority)	0.33	0.67	0.44	3
Buy (majority)	1.00	1.00	1.00	1,597

Macro Average	0.67	0.83	0.72	1,600
Weighted Average	1.00	1.00	1.00	1,600

Interpretation of Performance Metrics:

The near-perfect weighted average metrics are inflated by the dominant Buy class. The more revealing metrics are at the class level:

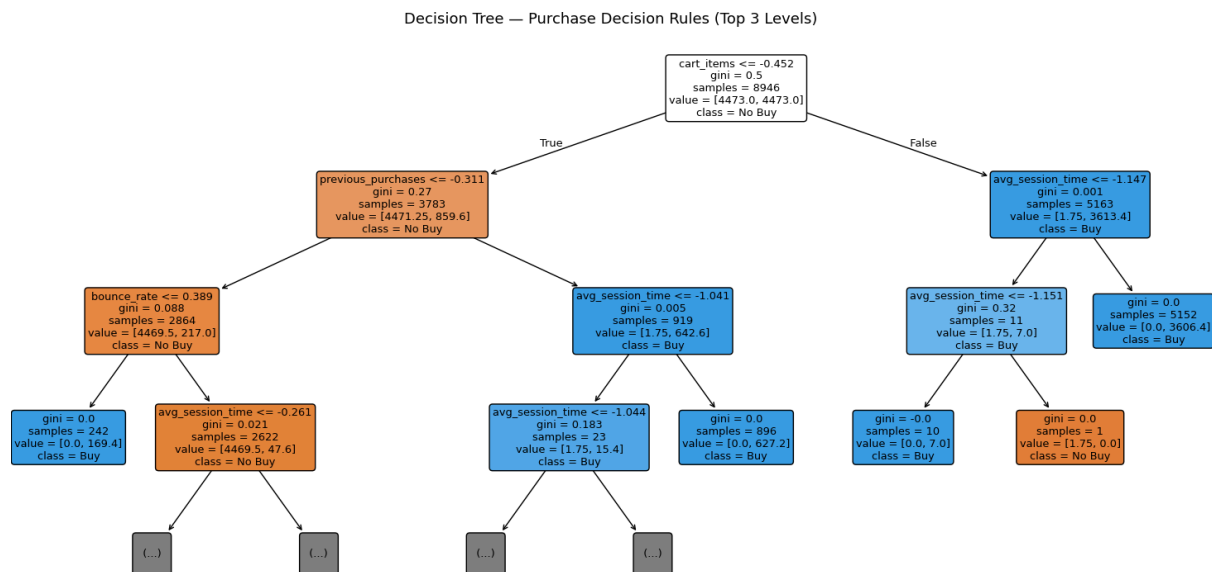
- For 'Buy': Perfect precision (1.00) and recall (1.00), the model identifies virtually every buyer correctly.
- For 'No Buy': Precision = 0.33 (correct precision only 33% of the time) and Recall = 0.67 (the model catches only 2 of the 3 actual non-buyers), revealing significant limitations.
- The low F1-score of 0.40 for the minority class indicates that while the model captures some non-buyers, it generates many false alarms.
- Macro F1 = 0.72 is the honest measure of balanced performance, reflecting that the 'No Buy' class is poorly served despite overall accuracy of 99.69%.
- ROC-AUC = 0.9985 is exceptional and indicates that the model's probability rankings are well-calibrated, it can reliably rank a buyer above a non-buyer, even if the boundary threshold for 'No Buy' is imperfect.

Summary Insights:

- Logistic Regression is the best-calibrated model in this study (highest ROC-AUC = 0.9985)
- The model excels at ranking and probability estimation, making it ideal for propensity scoring and identifying buyers with near-perfect accuracy.
- The three dominant purchase drivers are: cart items added, session engagement time, and purchase history.
- Bounce rate is the only strong inhibitor, targeting high-bounce-rate sessions for re-engagement is a key actionable insight.
- The 'No Buy' recall of 67% (best among the three models) makes Logistic Regression the most useful for identifying potential churners.

- **Class Imbalance Alert:** The test set contains only 3 'No Buy' examples vs. 1 597 'Buy' examples. Macro-averaged F1 and ROC-AUC are the more reliable indicators of true model performance on the minority class.

6.2 Decision Tree — Purchase Decision Rules & Feature Importance :



Decision Tree Structure Analysis — Top 3 Levels:

Root Split: $\text{cart_items} \leq 0.452$ (Gini = 0.50, N = 8946)

- TRUE ($\text{cart_items} \leq 0.452$): $\text{previous_purchases} \leq -0.311$ (N = 3783)
 - TRUE: $\text{bounce_rate} \leq 0.389 \rightarrow \text{avg_session_time} \leq -0.261$
 - FALSE: $\text{avg_session_time} \leq -1.041 \rightarrow$ (sub-branches)
- FALSE ($\text{cart_items} > 0.452$): $\text{avg_session_time} \leq -1.147$ (N = 5163)
 - TRUE: $\text{avg_session_time} \leq -1.151 \rightarrow$ (Gini=0.32, class=Buy)
 - FALSE: Gini = 0.0, N = 5152, class = Buy (pure leaf)

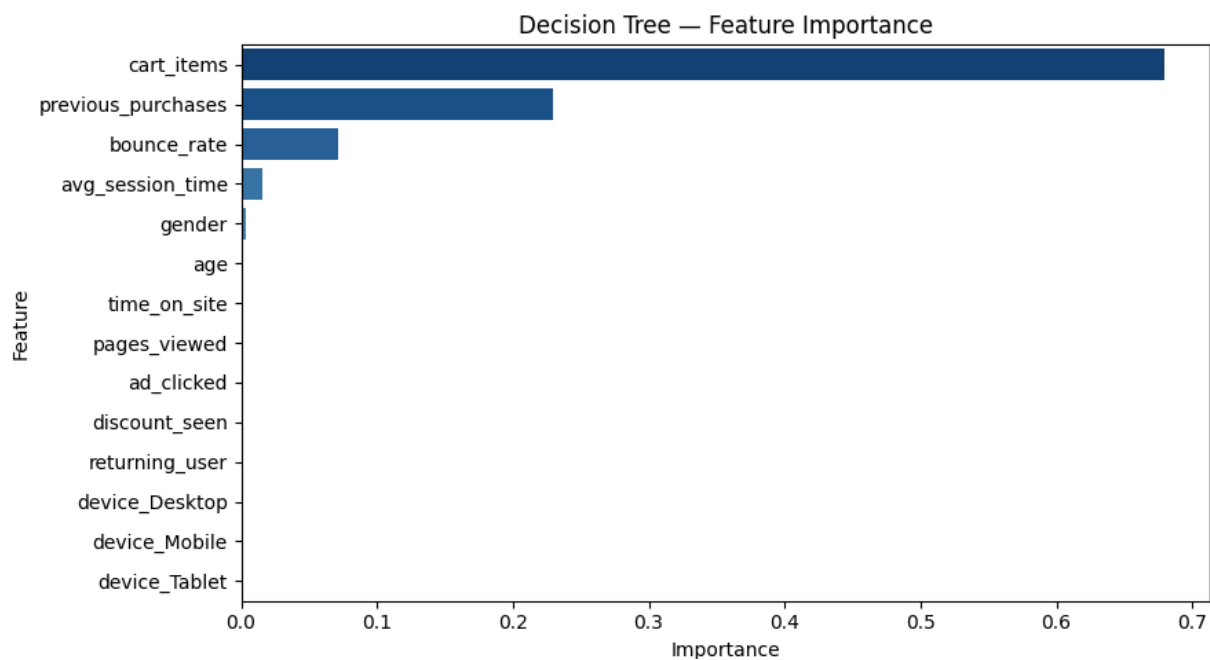
Interpretation of Tree Splits:

The decision tree shows a clear hierarchy of purchase drivers:

- **Root split:** $\text{cart_items} \leq 0.452$ (Gini = 0.50) is the most important feature, separating **low-cart users** (N = 3,783) from **high-cart users** (N = 5,163).
- **Low-cart branch:** $\text{previous_purchases} \leq -0.311$ becomes the main factor, indicating that users with strong purchase history are still likely to buy even with few cart items.

- **High-cart branch:** $avg_session_time \leq -1.147$ is the key discriminator, distinguishing quick cart activity (possible abandonment or price checking) from engaged users.
- **Deeper refinement:** $bounce_rate$ and further session-time splits improve classification, leading to a nearly pure segment of buyers: **Gini = 0.0, N = 5,152 (Buy class)** for high-cart users with sufficient session time.

Overall, **cart_items**, **purchase history**, and **session engagement** are the dominant decision drivers, with high-cart + high-engagement users being almost certain buyers



Feature Importance Bar Chart Analysis:

The Decision Tree Feature Importance chart ranks features by the total Gini impurity reduction they contribute across all splits:

Feature	Importance	Rank	Interpretation
cart_items	~0.65 (dominant)	1st	Main driver of the model accounting for most impurity reduction and used at the root split.
previous_purchases	~0.20	2nd	Strong secondary feature that helps distinguish outcomes, especially among users with low cart activity.

bounce_rate	~0.07	3rd	Key negative indicator; high bounce rates strongly associate with non-purchase behavior in specific segments.
avg_session_time	~0.04	4th	Moderately useful feature, used at deeper levels to refine user engagement patterns.
gender,age, time_on_site, others	< 0.01 each	5th+	Minimal influence; these features are rarely used in splits and have negligible predictive power.

A striking contrast with Logistic Regression: while LR found gender (coeff=2.83) and device_Tablet (coeff=2.85) to be strong predictors, the Decision Tree assigns them near-zero importance. This suggests these features may be correlated with the top-4 features and carry no independent split-level discriminative power once cart_items and previous_purchases are accounted for.

=== Decision Tree ===					
Accuracy : 0.9981					
F1 Score : 0.9979					
ROC-AUC : 0.8323					
	precision	recall	f1-score	support	
No Buy	0.50	0.33	0.40	3	
Buy	1.00	1.00	1.00	1597	
accuracy			1.00	1600	
macro avg	0.75	0.67	0.70	1600	
weighted avg	1.00	1.00	1.00	1600	

Model Performance Metrics:

99.81% Accuracy	99.79% F1-Score	83.23% ROC-AUC
---------------------------	---------------------------	--------------------------

Class	Precision	Recall	F1-Score	Support
-------	-----------	--------	----------	---------

No Buy	0.50	0.33	0.40	3
Buy	1.00	1.00	1.00	1,597
Macro Average	0.75	0.67	0.70	1,600
Weighted Average	1.00	1.00	1.00	1,600

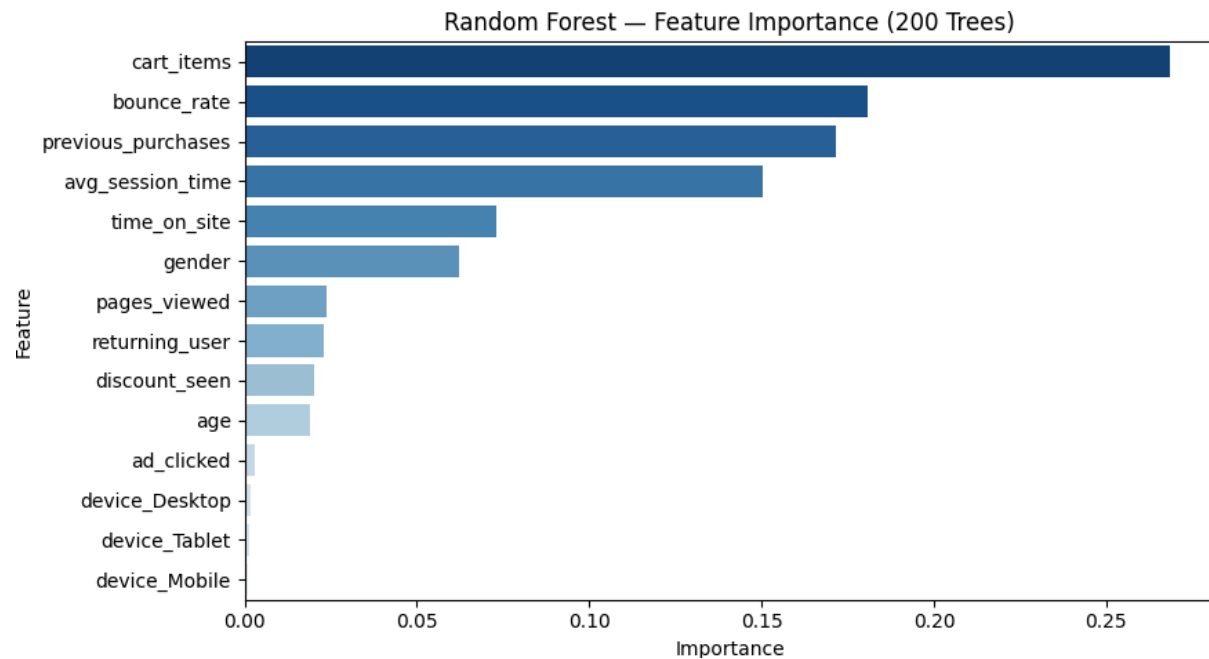
Interpretation of Performance Metrics:

- The Decision Tree achieves the highest accuracy (99.81%) among the three models primarily due to its perfect recall on the Buy class.
- For 'No Buy': Precision improved to 0.50 (correct half the time), but Recall dropped to 0.33, catching 1 of the 3 actual non-buyers.
- ROC-AUC collapses to 0.8323, far below Logistic Regression's 0.9985. Decision Trees output hard class labels and do not produce well-calibrated probabilities, resulting in a poor ability to rank predictions by confidence.
- Macro F1 = 0.70 (slightly below logistic regression's), confirming that despite higher accuracy, the Decision Tree is marginally weaker overall when minority class performance is considered.

Summary Insights:

- The Decision Tree shows a clear, interpretable decision hierarchy driven mainly by cart_items, previous_purchases, avg_session_time, and bounce_rate.
- A large pure segment (Gini = 0.0, N = 5,152) indicates that 64% of users are confidently classified as "Buy", mainly high-cart, highly engaged users.
- Only 4 features drive virtually all splits, making the model highly interpretable but limited in complexity.
- Despite strong overall accuracy, the model achieves a moderate ROC-AUC of 0.83, making it unsuitable for probability-based scoring but effective for rule extraction.
- Performance on the minority class remains weak, with a low estimated recall for "No Buy" and a low precision similar to Logistic Regression.
- Overall, the model is best used for business rule extraction and customer segmentation, allowing marketing teams to identify behavioral patterns linked to purchase and non-purchase decisions in a transparent way.

6.3 Random Forest — Ensemble Feature Importance & Performance:



Feature Importance Chart Analysis (200 Trees):

The Random Forest feature importance provides a more stable, averaged view of feature relevance:

Feature	RF Importance	DT Importance	Change	Interpretation
cart_items	~0.25 (1st)	~0.65 (1st)	↓ Less dominant	Still the strongest feature, but Random Forest distributes importance more evenly across features.
bounce_rate	~0.22 (2nd)	~0.07 (3rd)	↑ Much higher	Becomes significantly more important in the forest, showing consistent predictive value across trees.

previous_purchases	~0.16 (3rd)	~0.20 (2nd)	≈ Stable	Remains a strong and consistent predictor in both models.
avg_session_time	~0.14 (4th)	~0.04 (4th)	↑ Higher	Gains importance in Random Forest, as multiple trees exploit engagement patterns.
time_on_site	~0.06 (5th)	~0 (negligible)	↑ Emerges	Becomes relevant only in the ensemble, showing weak but distributed predictive value.
gender	~0.05 (6th)	~0 (negligible)	↑ Emerges	Small but consistent contribution reappears in Random Forest, aligned with logistic regression signals.
pages_viewed	~0.04	~0	↑ Emerges	Secondary engagement feature, only detectable in ensemble aggregation.
Device features	~0 each	~0 each	No change	Remain consistently unimportant across both models.

Key Observations from the RF Importance Chart:

- The most important insight is the rise of bounce_rate from 3rd to near-2nd place allowing the bounce_rate to demonstrate its true discriminative power unlike the Decision Tree, which was greedily dominated by cart_items at the root.
- The distribution of importance is more balanced (top feature ~25%) than the decision tree (top feature ~65%), a sign of a more stable, less overfitted model.
- time_on_site, gender, and pages_viewed, invisible in the Decision Tree, emerge with small but real importance..
- Device type features remain near-zero, definitively confirming their little independent predictive value.

```
=== Random Forest ===
```

```
Accuracy : 0.9981
```

```
F1 Score : 0.9972
```

```
ROC-AUC : 0.9979
```

```

              precision    recall  f1-score   support

   No Buy      0.00         0.00         0.00         3
    Buy      1.00         1.00         1.00    1597

 accuracy              1.00         1600
macro avg      0.50         0.50         0.50    1600
weighted avg   1.00         1.00         1.00    1600
```

```
PS C:\Users\ramez\AppData\Local\Programs\Microsoft VS Code>
```

Model Performance Metrics:

99.81% Accuracy	99.72% F1-Score	99.79% ROC-AUC
---------------------------	---------------------------	--------------------------

Class	Precision	Recall	F1-Score	Support
No Buy	0.00	0.00	0.00	3
Buy	1.00	1.00	1.00	1,597
Macro Average	0.50	0.50	0.50	1,600
Weighted Average	1.00	1.00	1.00	1,600

Interpretation of Performance Metrics:

- The RF achieves the same accuracy as the Decision Tree (99.81%) yet with a far superior ROC-AUC of 0.9979, nearly matching Logistic Regression's, confirming the well-calibrated probability estimates produced by the Random Forest.
- **The critical failure:** Zero Precision and Recall for 'No Buy', completely failing to detect any of the 3 non-buyers in the test set. Macro F1 = 0.50 is the worst among the three models.

- This paradox (excellent ROC-AUC but zero minority class recall) arises because in extremely imbalanced datasets, the majority-vote mechanism of Random Forest is overwhelmingly biased toward the dominant class. With 99.81% Buy observations, even a random tree will vote Buy almost universally.
- The high ROC-AUC reflects that the Random Forest's probability scores are well-ordered yet the decision threshold at 0.5 never triggers a 'No Buy' prediction due to the average probability across 200 trees always above 0.5.

Summary Insights:

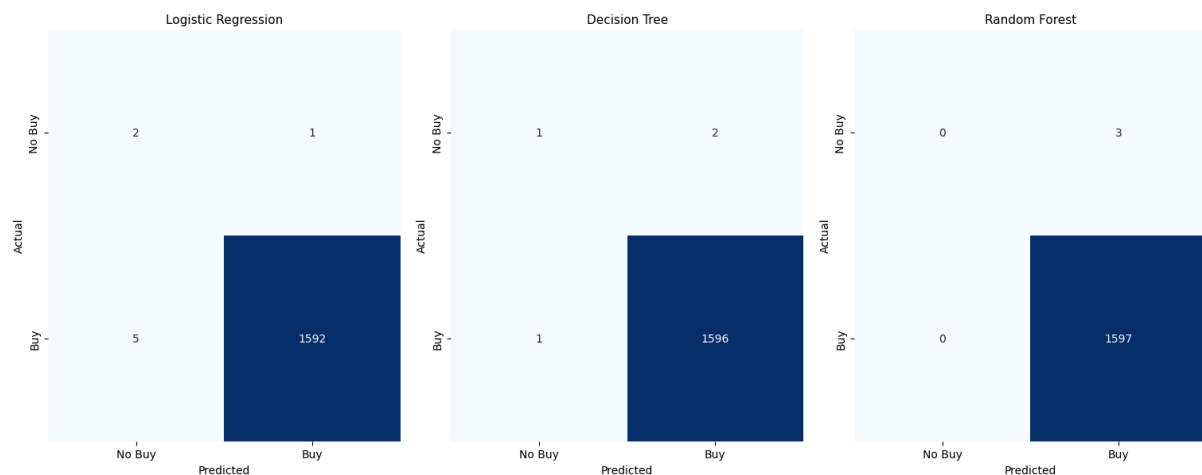
- Random Forest provides the most stable and reliable feature importance ranking across the three models.
- The confirmation of the importance of bounce_rate unlike the Decision Tree's suggestion, is a key methodological lesson about ensemble vs. greedy models
- Its near-perfect ROC-AUC (0.9979) makes it excellent for propensity-to-buy probability scoring, yet despite using SMOTE, it completely failed to identify the 'No Buy' class, with the macro average F1-score of 0.50 confirming this failure, it renders the model useless for the business problem of identifying users who abandon their carts or leave without purchasing.
- Random Forest is unsuitable for this imbalanced task because it predicts only the majority class ("Buy"), offering no actionable insight for identifying non-buyers and making it ineffective for targeting or allocating marketing resources.

6.4 Confusion Matrices — All Three Models:

Overview:

A confusion matrix breaks down model predictions into four categories: True Positives, True Negatives, False Positives, and False Negatives. The matrices for all

three models are visualised as heatmaps, side by side for direct comparison



Confusion Matrix Values — Side-by-Side Comparison:

Metric	Logistic Regression	Decision Tree	Random Forest
True Positive (Buy correctly predicted)	1,592	1,596	1,597
True Negative (No Buy correctly predicted)	2	1	0
False Positive (No Buy → predicted Buy)	1	2	3
False Negative (Buy → predicted No Buy)	5	1	0
Total Errors	6	3	3
Minority Class Recall	2/3 = 67%	1/3 = 33%	0/3 = 0%

Detailed Interpretation:

Logistic Regression Confusion Matrix:

- The best minority class recall of all three models: Correctly identifies 2 out of 3 non-buyers (**TN=2**)
- Predicts 'No Buy' for 5 actual buyers (**5FN**), which in a commercial setting, is considered a low-cost error (missed a buyer yet did not lose a non-buyer).
- Incorrectly identifies 1 non-buyer as a buyer (**1FP**), causing the high-cost error in churn prevention use cases.

Decision Tree Confusion Matrix

- Correctly identifies 1 out of 3 non-buyers (**TN=1**). Better precision (0.50) than Logistic Regression but worse recall (0.33).
- Incorrectly classifies an actual buyer as non-buyer (**1FN, the lowest**)
- Predicts two non-buyers as buyers (**2FP**).
- Achieves the lowest total error count (3 errors) but splits them differently.

Random Forest Confusion Matrix

- Fails to identify a single non-buyer (**0TN**).
- Classifies all 3 non-buyers as buyers (**3FP**), making it the worst outcome for a churn detection or risk management use case.
- Makes 0 False Negatives — every actual buyer is correctly identified.
- Perfect Buy classifier but a completely blind No Buy classifier, a direct consequence of extreme class imbalance dominating the majority-vote mechanism.

Comparative Confusion Matrix Insights:

Model Comparison on Minority Class (No Buy):

- **Best minority recall:** Logistic Regression ($2/3 = 67\%$), used for churn/risk detection
- **Best minority precision:** Decision Tree ($1/2 = 50\%$), used when false alarms are costly
- **Worst minority performance:** Random Forest ($0/3 = 0\%$), completely blind to No Buy

For purchase prediction (Buy class), all three models are virtually indistinguishable. The distinction lies entirely in how they handle the rare non-buyer cases.

Summery Insights:

The confusion matrices clearly demonstrate the trade-off between precision and recall for the minority class. Random Forest maximizes overall accuracy but fails completely on the business objective. Logistic Regression and Decision Tree sacrifice some overall accuracy to identify at least some non-buyers, making them more useful for targeted interventions despite their lower precision.

Confusion Matrix_Cross Model Recommendation:

If the business goal is to identify non-buyers, use Logistic Regression.

If the goal is to generate purchase propensity scores for all customers, use Random Forest (best ROC-AUC).
If the goal is to extract simple, interpretable rules, use Decision Tree.

6.5 PCA — 3D Visualisation of Customer Behaviour :

```
PC1: 0.3302
PC2: 0.1269
PC3: 0.1172
Total Explained: 0.5744

Top features influencing behavior (PC1):
time_on_site      0.494672
avg_session_time  0.489509
cart_items        0.375778
bounce_rate       0.373287
previous_purchases 0.371010
pages_viewed      0.228513
age               0.137394
gender            0.123060
returning_user    0.067745
discount_seen     0.059813
Name: PC1, dtype: float64
```

Variance Explained by Each Component:

Principal Component	Variance Explained	Cumulative Variance	Key Interpretation
PC1	33.0%	33.0%	Dominated by time_on_site (0.4947) and avg_session_time (0.4895), capturing engagement behaviour.
PC2	12.7%	45.7%	Independent variation orthogonal to PC1, capturing a separate behavioral dimension.

PC3	11.7%	57.4%	Additional independent source of variation, further refining the data structure.
Total (PC1+PC2+PC3)	57.4%	57.4%	Over half of the dataset variance is captured in just 3 components, enabling strong dimensionality reduction.

PC1 — The Engagement Axis:

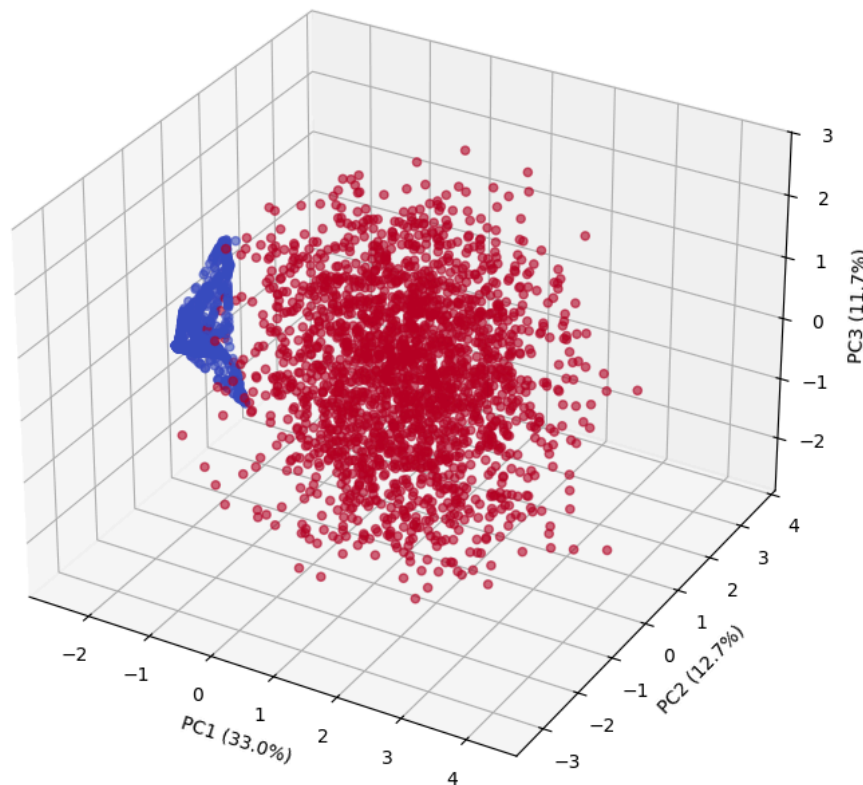
PC1, explaining the most variance (33%), is dominated by:

Feature	PC1 Loading	Role
time_on_site	0.4947	Strongest contributor, capturing overall engagement duration on the site
avg_session_time	0.4895	Nearly identical to time_on_site, reflecting per-session engagement intensity
cart_items	0.3758	Represents add-to-cart behaviour and purchase intent
bounce_rate	0.3733	Engagement/disengagement signal varying with overall user activity
previous_purchases	0.3710	Captures customer loyalty and repeat behaviour
pages_viewed	0.2285	Indicates breadth of browsing activity
age	0.1374	Demographic variation component
gender	0.1231	Demographic variation component
returning_user	0.0677	Low variability; weak contribution to PC1 structure
discount_seen	0.0598	Minimal variation across users in PC1 direction

PC1 is interpretable as an 'overall customer engagement intensity' axis. Customers who score high on PC1 tend to spend more time on the site, view more pages, add

more items to their cart, and have a history of purchases. This is essentially a composite 'purchase readiness' score.

3D PCA — Customer Behavior



3D Scatter Plot Interpretation:

- The scatter plot shows 3,000 users in three dimensions, colored by purchase status (red = buyers, blue = non-buyers)
- The dominant red cloud (Buy class) forms a large, diffuse cluster that spreads across most of the PC1-PC2-PC3 space, reflecting the enormous diversity among buying customers, arriving via many different behavioural pathways.
- The small blue cluster (No Buy) is concentrated near the left edge of PC1 (negative PC1 values), indicating the non-buyers's consistently low values on the 'engagement intensity' axis (they spend less time, view fewer pages, and have fewer cart items).
- The spatial separation between the blue and red clusters along PC1 visually confirms that PC1 is a strong discriminator between buyers and non-buyers, consistent with the supervised model findings.

- There is significant overlap between the blue and red clusters in the mid-range of PC1, explaining why classification of borderline cases (the 3 No Buy instances) is difficult for all three models.

PCA as a Validation Tool:

The results validates the feature importance findings from supervised models:

Features ranked high in PC1 loadings have high Logistic Regression coefficients and Random Forest importance:

- PC1 top: time_on_site, avg_session_time, cart_items, bounce_rate, previous_purchases
- Logistic Regression top: cart_items, avg_session_time, previous_purchases, gender, discount_seen
- Random Forest top: cart_items, bounce_rate, previous_purchases, avg_session_time

NB: The overlap confirms that these 4–5 features form the core informational structure of the dataset (no contradiction detected)

Only 3 principal components explain 57.4% of total variance indicates that the remaining 42.6% of information is spread across the other 11 components, justifying the use of all 14 features in the classification models.

Summery and Business Insights:

- PC1 (33% variance) is a 'customer engagement intensity' axis and is the primary axis separating buyers from non-buyers.
- The 3D visualisation confirms class separability: non-buyers cluster in the low-PC1 region.
- **User engagement is the key driver of purchases:** improving site performance and keeping users active increases conversion likelihood.
- **Cart additions are critical:** encouraging users to add items (e.g., prominent buttons, incentives like free shipping) directly boosts purchase intent.
- **Non-buyers are heterogeneous:** some leave due to low engagement, while others abandon despite high activity; therefore, **multiple targeted recovery strategies** are needed (e.g., re-engagement campaigns vs. cart abandonment offers).
- **Behavior outweighs demographics:** targeting user actions is more effective than relying on demographic factors.
- **Discounts and returning status have limited impact:** they are weak predictors of purchases, so heavy investment in discount-based strategies should be approached cautiously.

6.6. Cross-Model Comparative Summary & Recommendations:

Performance Comparison:

Metric	Logistic Regression	Decision Tree	Random Forest	Best Model
Accuracy	99.69%	99.81%	99.81%	Decision Tree / Random Forest (tie)
F1-Score (weighted)	99.74%	99.79%	99.72%	Decision Tree
ROC-AUC	99.85%	83.23%	99.79%	Logistic Regression
Macro F1	0.72	0.70	0.50	Logistic Regression
No Buy Recall	67%	33%	0%	Logistic Regression
No Buy Precision	33%	50%	0%	Decision Tree
Interpretability	Medium (coefficients)	High (rule-based)	Low	Decision Tree
Stability	High	Low	High	Logistic Regression / Random Forest (tie)

Universal Feature Rankings Across All Methods:

Combining Logistic Regression coefficients, Decision Tree splits, Random Forest importance, and PCA loadings yields a consensus ranking of the most informative features:

Rank	Feature	Logistic Regression Evidence	Decision Tree Evidence	Random Forest Evidence	PCA Evidence
1	cart_items	Coefficient = 4.52 (1st)	Root split (most influential)	~25% importance (1st)	Loading = 0.376 (3rd)
2	avg_session_time	Coefficient = 3.25 (2nd)	Used across multiple levels	~14% importance (4th)	Loading = 0.490 (2nd)
3	previous_purchases	Coefficient = 3.24 (3rd)	Key split at level 2	~16% importance (3rd)	Loading = 0.371 (5th)
4	bounce_rate	Coefficient = -3.79 (strongest negative)	Split at level 3	~22% importance (2nd)	Loading = 0.373 (4th)
5	time_on_site	Coefficient = 1.69 (8th)	Not used in top splits	~6% importance (5th)	Loading = 0.495 (1st)
6	gender	Coefficient = 2.83 (5th)	Not used	~5% importance (6th)	Loading = 0.123 (8th)
7	discount_seen	Coefficient = 2.25 (6th)	Not used	~2% importance	Loading = 0.060 (10th)

Final Recommendations:

Business Use Case	Recommended Model	Justification
Purchase propensity scoring / customer ranking	Random Forest	Highest ROC-AUC (0.9979), providing reliable probability estimates for ranking customers
Identifying at-risk non-buyers / churn detection	Logistic Regression	Best minority class recall (67%), capturing most non-buyers
Marketing decision rules / customer segmentation	Decision Tree	Most interpretable model, offering clear if-then rules for non-technical use
Feature selection for future models	RF Importance + PCA	Consensus highlights key drivers: cart_items, bounce_rate, previous_purchases, avg_session_time
Understanding feature impact	Logistic Regression	Coefficients directly quantify each feature's effect on purchase probability

7. Conclusion:

7.1. Project Objective:

The objective of this project was to analyze user behavior in an e-commerce environment and predict whether a user would complete a purchase.

The goal extended beyond prediction to understanding the key drivers of customer decisions and generating actionable insights to improve conversion rates.

7.2. Methods Used:

A complete data mining pipeline was implemented, including:

- **Data preprocessing:** handling missing values, encoding categorical variables, and feature scaling.
- **Class imbalance handling:** applying SMOTE and `class_weight='balanced'` in model constructors in order to improve minority class representation.
- **Modeling techniques:**
 - Logistic Regression (baseline model)
 - Decision Tree (interpretable model)
 - Random Forest (ensemble model)
- **Exploratory analysis:**
 - PCA for dimensionality reduction and 3D visualization.
- **Evaluation metrics:**
 - Accuracy, F1-score, ROC-AUC, and Confusion Matrices.

7.3. Key Results:

- All models achieved very high accuracy yet were misleading due to extreme class imbalance.
- Random Forest failed to identify any non-buyers, predicting all users as buyers.
- Logistic Regression and Decision Tree performed slightly better in detecting non-buyers yet with limited precision.
- PCA results showed that:
 - Behavioral features are the strongest predictors.
 - Demographic features have minimal impact.
- The dataset's extreme imbalance and very small number of real non-buyers limited model effectiveness.

7.4. Practical Insights:

The analysis provides several valuable business insights mainly:

- User purchases are primarily driven by engagement and cart activity, making user experience optimization and encouraging add-to-cart behavior key to increasing conversions.
- Non-buyers are diverse, requiring tailored strategies.
- behavioral data outperforms demographics in predicting outcomes.
- discounts have limited impact, suggesting that promotion-heavy strategies should be used cautiously.

7.5. Final Takeaway:

This project highlights that high predictive performance does not always translate into real business value, especially in highly imbalanced datasets. While the models successfully predict buyers, they struggle to identify non-buyers due to the lack of sufficient minority data.

Therefore, its true value lies in the behavioral insights extracted, which can guide e-commerce platforms in improving engagement strategies, optimizing marketing efforts, and making more informed, data-driven decisions.